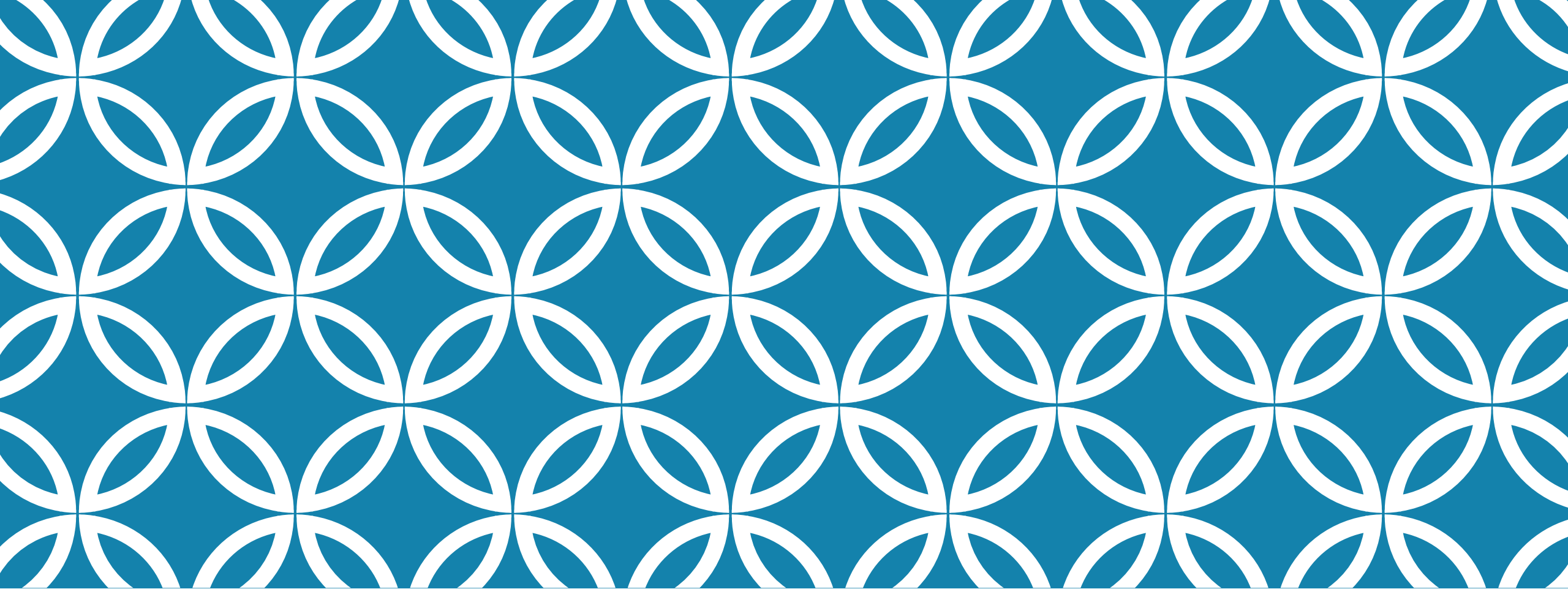




STATIC PROGRAM ANALYSIS

Lecture 15 – Abstract Interpretation

VALUE-BASED ANALYSIS

Previous analysis focus on questions that require def-use relations

- Reaching Definitions $\{x@2, y@3\}$
- Available Expressions, Very Busy Expressions $\{x+y, 2x\}$
- Live variables $\{x, z\}$

$\hat{\sigma}$ is an abstract state

Some problems require to know possible values of variables

- Constant propagation optimization
- Assertion violation
- Post condition
- More precise dead code elimination

IN/OUT propagates

- A map $\hat{\sigma}: Var \rightarrow Val$ or some function over variables $F(v_1, v_2, v_3, \dots, v_n)$
- Forward may analysis
- Transfer function cannot be decomposed into gen/kill parts – more sophisticated computation

CONSTANT PROPAGATION

IN/OUT (or state)

- $\hat{\sigma}: (Var \rightarrow Z_{\perp}^T)$
- $\hat{\sigma}(x) \rightarrow 1$ var x is a constant 1
- $\hat{\sigma}(y) \rightarrow \top$ var x is not a constant
- $\hat{\sigma}(z) \rightarrow \perp$ var z is unknown yet

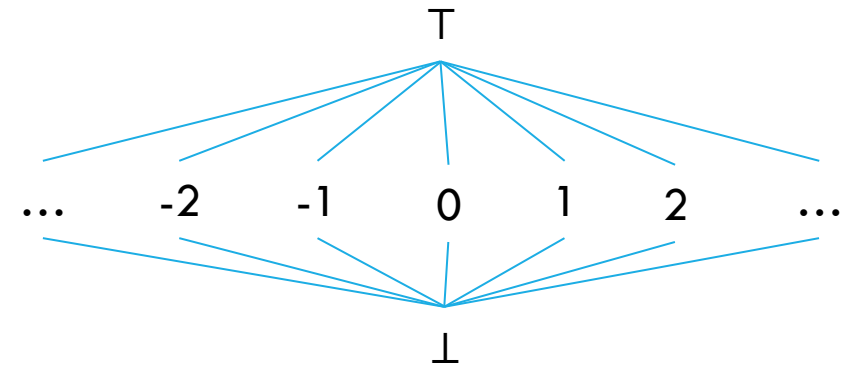
On merges

- $(\hat{\sigma} \sqcup \hat{\sigma})(z) = \hat{\sigma}(z) \sqcup \hat{\sigma}(z)$ as described in the lattice

Transfer function $c \in Z$

- assignment $x := c$
 - $f^{\{x:=c\}}(\hat{\sigma}) = \hat{\sigma}[x \rightarrow c]$
- branches $x == c$
 - $f_t^{\{x==c\}} = \hat{\sigma}_t[x \rightarrow c]$,
 - $f_f^{\{x==c\}} = \hat{\sigma}_f[x \rightarrow \top]$
- others:
 - $f(\hat{\sigma}) = \hat{\sigma}$

state is updated for variable x to a constant c



$$\begin{aligned} 1 \sqcup 1 &= 1, 1 \sqcup 0 = \top, \\ 1 \sqcup \perp &= 1, 1 \sqcup \top = \top \end{aligned}$$

PROPERTIES OF ABSTRACT DOMAINS

All possible concrete values of a variable should be described by an element of abstract domain.

- $x = 1 \vee x = 2 \vee x = 5$ (possible values of variable x at location k)
- $\hat{\sigma}(x) \rightarrow \top$
- Could include extra
- Abstract elements must cover the entire domain, e.g., $x = 1$ has no corresponding abstract element describing it in a domain D then D is not an abstract domain.

Also requires several relationships between values produced during concrete computations vs. values produced during analysis computation.

Transfer function values should monotonically increase to ensure fixed-point computation

- Already is given for abstract domains with finite complete lattices

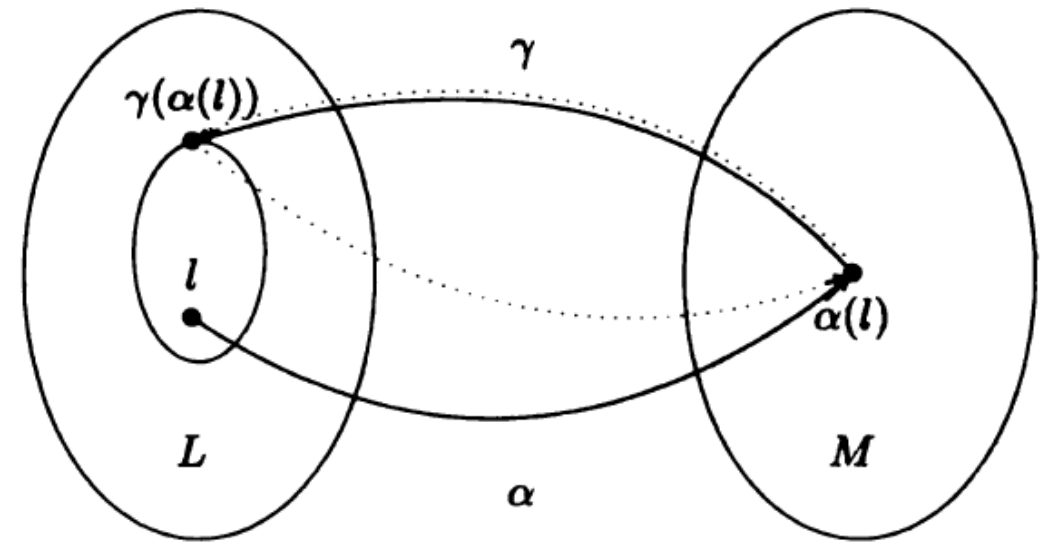
PROPERTY 1: RELATION BETWEEN CONCRETE AND ABSTRACT VALUES

Let L be a concrete lattice, e.g., $\wp(Z)$

Let M be an abstract lattice, e.g., $Sign$

They are connected by two monotone functions:

- Abstraction $\alpha: L \rightarrow M$
- Concretization $\gamma: M \rightarrow L$
- $\forall l \in L \gamma(\alpha(l)) \supseteq l$
- $\forall m \in M \alpha(\gamma(m)) \sqsubseteq m$



PROPERTY 2: RELATION BETWEEN TRANSFER FUNCTIONS

Transfer function for concrete lattice

$$f: L \rightarrow L$$

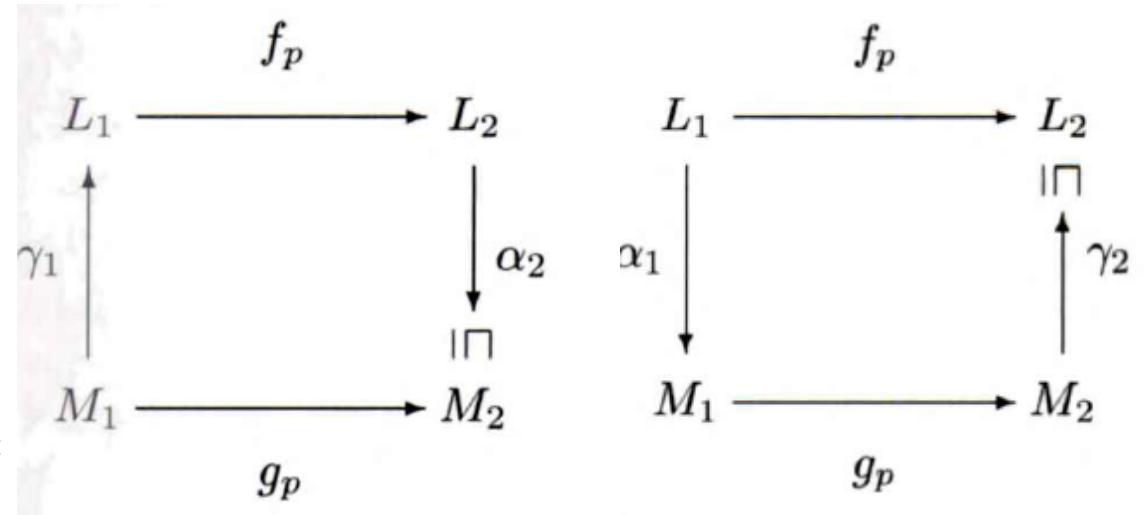
Transfer function for abstract lattice

$$g: M \rightarrow M$$

Relation between them for any statement p:

$$\alpha(f(\gamma(m))) \sqsubseteq g(m)$$

$$\gamma(g(\alpha(l))) \sqsupseteq f(l)$$



Most precise abstract transfer function:

$$\alpha(f(\gamma(m))) = g(m)$$

PROPERTY 3: ENSURING TERMINATION

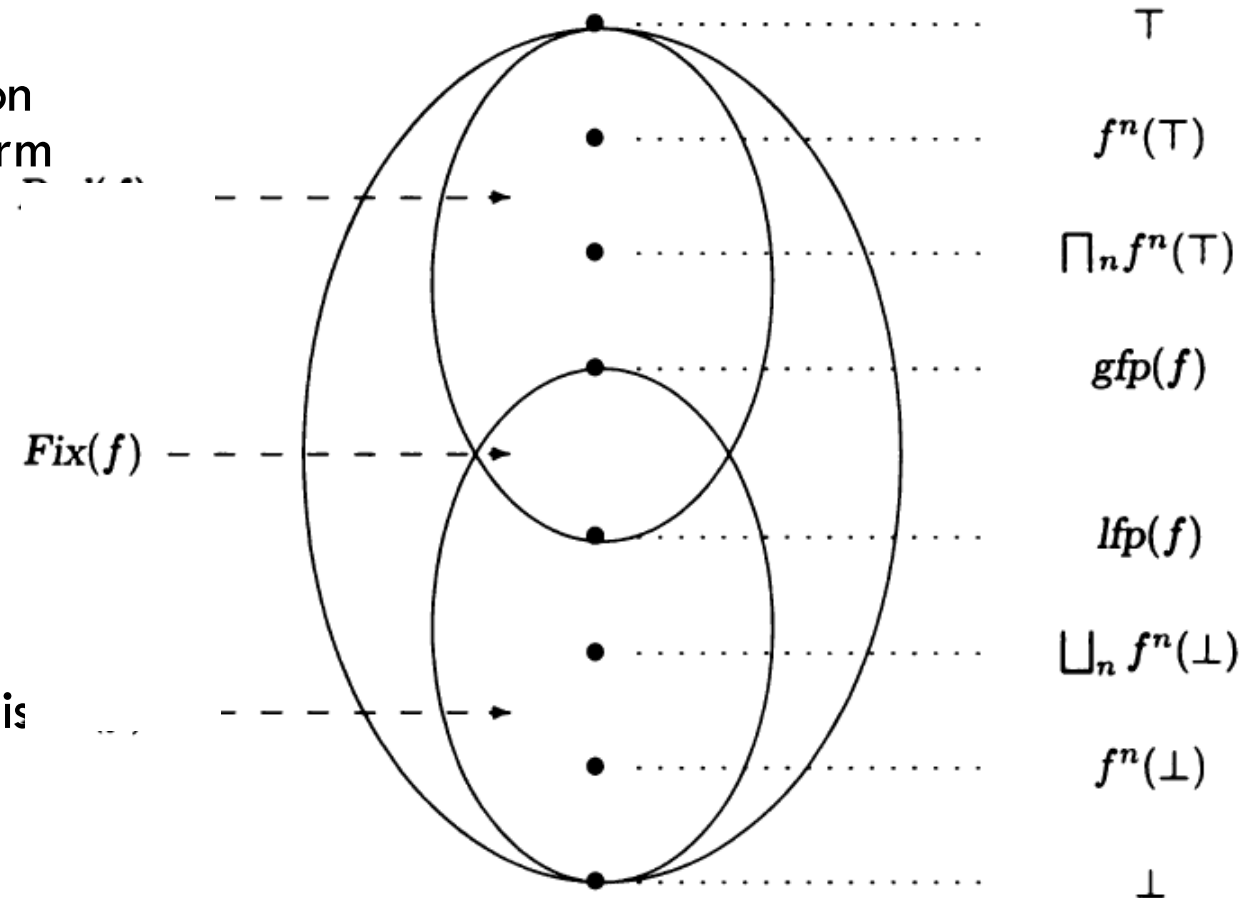
Function f is a monotone function, which depends on the program P and describes how one fact transform into another fact from a lattice

- $L = (L, \sqsubseteq, \sqcup, \sqcap, \top, \perp)$
- $f(l_1) = f(l_2)$
- $f: L \rightarrow L$

Monotone: if l'_1 describes at least the value that l_1 does then $f(l'_1)$ describes at least the value that $f(l_1)$ does.

Fixed point of a function f when computing analysis

- $Fix(f) = \{l \mid f(l) = l\}$, and $Fix(\perp)$ is the actual solution
- $gfp(f)$ is what fixed-point algorithm computes.



PROPERTY 3: ENSURING TERMINATION

In complete finite lattices gfp is guarantee to be reached and be the same as $Fix(\perp)$.

- Predicate
- Set of variables
- Set of expressions

Can relax finite requirement and ask for it has infinite ascending chains that eventually stabilizes.

Thus, we can select an upper bound value for those chains.

Need to have a widening operator

- Interval
- Relational domains

